Sako Supply: A Web-Based Wholesale Rice Ordering and Inventory Management System

An E-Commerce Web System Presented to
the Faculty of the Information Technology Department
St. Alexius College

In Partial Fulfillment of the Requirements in
IT 201 – Platform Technologies

By
Hameem N. Abdulhakim
[Course and Year]

[Instructor / Adviser]
October 2026

Document Status and Reading Guide

This document connects the research problem, objectives, methodology, system design, implemented modules, and expected deliverables of Sako Supply. It follows the supplied research outline and is prepared as an editable draft for an individual academic project.

The software discussion is based on the project repository (Abdulhakim, 2026), checkpoint e711837, validated in the development environment on October 24, 2026. The implementation record reports 19 passing backend/security tests and 11 passing browser tests. These records establish specific software behaviors; they do not establish business effectiveness, completed user acceptance testing, or participant satisfaction.

No interview transcripts, survey responses, signed evaluation forms, or verified partner-store information were supplied for this draft. Consequently, field requirements gathering and user evaluation are presented as planned procedures. The researcher must replace the course/year, adviser, study locale, schedule, and any empirical evidence placeholders before treating the document as a final submission.

Chapter I defines the study and five objectives. Chapter II explains how those same objectives are translated into requirements, architecture, database relationships, and evaluation procedures. Chapter III identifies the corresponding modules, the indicative work schedule, and the deliverables. The references distinguish documented existing systems from proposed local research activities.

CHAPTER I
INTRODUCTION

1.1 Background of the Study

A bigasan supplies rice for household consumption and for customers who purchase larger quantities for food preparation, resale, or community activities. In a wholesale-oriented transaction, the buyer needs clear information about the rice type, the number of sacks available, the price per sack, and the arrangement for pickup or delivery. These details are related: a change in quantity can affect the applicable bulk discount, while the selected delivery area can change the final amount due.

When order details, stock counts, and payment records are maintained separately, a store may find it difficult to reconcile which sacks are available for a new buyer and which have already been reserved. A customer may also need to contact the seller repeatedly to confirm an order or obtain a transaction record. These are plausible workflow concerns that motivate the prototype; they are not presented as measured conditions at a named store. Their actual occurrence and importance must be established through the requirements-gathering procedure in Chapter II.

Existing platforms illustrate that online selling can be connected to inventory and administrative work. WooCommerce provides a customizable WordPress-based commerce platform, while ERPNext describes order management and stock-related business processes. Odoo combines web-based business applications, including e-commerce and warehouse management, and Medusa organizes commerce capabilities into customizable modules (WooCommerce, n.d.; Frappe Technologies, n.d.; Odoo S.A., n.d.; Medusa, n.d.). These systems establish useful design references, but they do not determine the precise requirements of an individual rice-store prototype.

Sako Supply is therefore developed as a focused web-based prototype for one bigasan selling 25-kilogram sacks. The customer-facing website supports product details, account access, bulk quantities, checkout, order history, and printable order records. The administrative interface supports product images, stock movements, configurable reorder points, order processing, cash-collection entries, charts, and an audit log. The system calculates wholesale discounts and sample delivery-zone fees on the server instead of trusting a price supplied by the browser.

The prototype deliberately separates implemented software behavior from real commercial operations. Card, GCash, and Maya checkout outcomes are simulations; no real electronic payment is processed. Delivery visualization uses an illustrated local map and an updating sample route rather than a live rider-location feed. This boundary makes the project manageable for an individual researcher and allows the evaluation to focus on correct ordering, inventory control, transaction visibility, and understandable navigation.

The central concern of this study is consequently whether a focused, integrated prototype can support a coherent wholesale rice-ordering workflow. Chapter II translates this concern into requirements and tests, while Chapter III identifies the modules and outputs through which the objectives can be demonstrated.

1.2 Statement of the Problem

The study seeks to address the design and evaluation of a web-based wholesale rice ordering and inventory management prototype. Specifically, it is guided by the following questions. These questions must be validated against the actual needs of the intended users; they do not imply that interviews or observations have already established the problems.

RQ1. How can customers obtain clear rice-product information and place 25 kg sack orders with consistent bulk prices and delivery-zone fees?

RQ2. How can the system reserve available stock during checkout, prevent overselling, and restore reserved quantities when an eligible order is cancelled?

RQ3. How can customers review their own orders, obtain printable transaction records, and understand order progress without mistaking simulated payment or delivery information for a real service?

RQ4. How can the administrator monitor products, stock changes, order processing, recorded cash collections, and administrative actions in one interface?

RQ5. How can the prototype be evaluated for functional correctness, reliability, basic security, and ease of use using documented tests and a planned user evaluation?

1.3 General Objective

To develop and evaluate Sako Supply, a web-based prototype that integrates 25 kg wholesale rice ordering, inventory reservation, customer transaction visibility, and administrative monitoring for a single bigasan.

1.4 Specific Objectives

SO1. To implement an authenticated rice catalog and ordering workflow that applies server-calculated bulk discounts and delivery-zone fees.

SO2. To implement inventory reservation, validated stock adjustments, configurable reorder points, and cancellation-based stock restoration.

SO3. To provide account-specific order history, printable order records, status updates, reorder functionality, and clearly identified delivery and payment simulations.

SO4. To provide administrative product management, order processing, sales charts, a cash-collection ledger, and an audit trail.

SO5. To verify the prototype through documented automated and functional checks and to prepare a task-based user evaluation of the completed workflow.

Table 1.1. Alignment of the research questions and objectives

Question	Objective	Evidence to be produced
RQ1	SO1	Catalog-to-checkout demonstration; tier and zone calculation cases
RQ2	SO2	Stock reservation, cancellation, SQL validation, and last-sack race tests
RQ3	SO3	Ownership checks; receipt, status, reorder, and simulation-label checks
RQ4	SO4	Dashboard, stock history, chart, cash-ledger, and audit demonstrations
RQ5	SO5	Test records and planned task-based UAT instrument

1.5 Scope and Limitations

1.5.1 Scope

The study covers a single-store prototype with two application roles: customer and configured administrator. All active rice products are sold as 25 kg sacks, and customers may order multiple sacks without an imposed minimum sack count. Dinorado, Jasmine, Sinandomeng, well-milled white rice, and brown rice are represented as sample Philippine-market rice retail descriptions. They are not all commercial brands or certified cultivars. Supplier specifications, commercial availability, origin, prices, packaging, and image permissions require confirmation before any actual selling activity.

Customers can register and sign in, maintain basic contact information, search and filter the catalog, read product details, order rice, view their own orders, print an order acknowledgment or receipt, repeat an order using current prices and stock, and cancel their own orders while Pending. Delivery orders include a sample zone choice and an illustrated progress map. In-app notifications record order-related updates without sending messages externally.

The administrator can create, edit, or archive products; upload or replace product images; record stock adjustments with reasons; set reorder points; process orders; and record cash collection. The dashboard includes daily and weekly completed-order charts, while the reporting interface distinguishes completed-order totals from recorded cash collections. An audit log records successful administrative mutations under the configured admin identity.

The implementation uses HTML, CSS, and browser JavaScript, a Node.js backend, and an SQLite SQL database. Local operation on a Windows laptop is the primary demonstration workflow. The study includes a basic public app-shell cache, database/upload backup utilities, and a health endpoint. Technical validation is limited to the scenarios actually executed and recorded.

1.5.2 Limitations

Electronic payments are simulated. The card fields demonstrate successful or declined test outcomes, and their card number, expiry, and CVC values are not submitted to the application server. GCash and

Maya are demonstration choices rather than connected merchant services. Printed documents are research-demo transaction records, not BIR official receipts or tax invoices.

The delivery map uses a bundled illustration and a fixed sample route. Its position updates are generated by the prototype; they are not driver GPS, address geocoding, actual road routing, or a guarantee of delivery to the selected zone. Sample zone charges are Koronadal ₱150, Tupi ₱300, and Tantangan ₱250. These are configuration examples rather than verified logistics quotations.

The study does not include multiple stores, cashier/staff permissions, customer credit and partial-payment accounting, cost-based profit calculation, real email/SMS delivery, password-reset delivery, verified-email registration, multiple saved addresses, or complete Filipino/Bisaya localization. These are possible future extensions. The basic PWA shell does not permit offline sign-in, checkout, or tracking.

Sales charts represent completed orders grouped by order-placement date in Philippine time and may include simulated electronic payments. The cash ledger represents admin-recorded cash collections grouped by collection date; it is not independent confirmation that money physically changed hands. Historical Paid entries without a collection record are not reconstructed. No participant acceptance score, business benefit percentage, ISO certification, or production security certification is claimed.

1.6 Significance of the Study

For a prospective store owner or administrator, the prototype provides a concrete basis for examining whether ordering and stock records can be coordinated in one workflow. Stock histories, reorder points, and audit entries may improve the visibility of recorded actions, but any reduction in errors or processing time must be measured rather than assumed.

For customers, the website demonstrates access to product information, calculated prices, personal order records, and understandable status information. The proposed task-based evaluation will determine whether intended users can complete these actions without assistance and whether the simulation labels prevent misunderstanding.

For the researcher, the project provides an individual application of platform technologies, database design, client–server interaction, role-based access checks, and software testing. For instructors and future researchers, the documented architecture, requirements traceability, and identified limitations provide a reproducible reference for evaluating or extending a focused academic prototype.

1.7 Review of Related Literature and Existing Systems

This section reviews four documented existing systems and supporting technical guidance. It is a documentary comparison, not a hands-on benchmark or a claim that the researcher has conducted empirical studies of these products. Official project material was used to establish general capabilities. No local thesis, interview finding, or statistical result is invented to fill a literature requirement.

1.7.1 WooCommerce

WooCommerce describes an open-source commerce platform for WordPress with customizable storefronts, product pages, shopping-cart experiences, and optional payment and delivery integrations. Its official product readme also describes administrative performance monitoring and extension-based customization (WooCommerce, n.d.). The relevance to Sako Supply is the connection between product presentation, checkout, and store management. Sako Supply uses a smaller custom implementation for a predefined 25 kg rice workflow rather than implementing the full WordPress and extension ecosystem.

1.7.2 Odoo

Odoo presents a suite of web-based business applications that includes e-commerce, warehouse management, accounting, and point of sale. Its official repository explains that applications can operate separately or be integrated into an ERP arrangement (Odoo S.A., n.d.). This supports the principle that customer-facing and administrative processes can share consistent business data. Sako Supply adopts that integration principle at prototype scale while excluding the broader accounting, personnel, and multi-application scope.

1.7.3 ERPNext

ERPNext identifies accounting and order management among its major features. Its order-management description includes tracking inventory, replenishing stock, managing sales orders, and coordinating fulfillment (Frappe Technologies, n.d.). These functions are directly relevant to the relationship between stock availability and customer orders. In Sako Supply, the related functions are limited to stock reservation, adjustment history, order-status progression, and a recorded cash ledger; they do not constitute a complete ERP or accounting system.

1.7.4 Medusa

Medusa describes a modular commerce platform that can support business-to-business stores and distributor applications through commerce primitives such as products, carts, orders, pricing, and payments. Its repository separates core modules, API routes, and infrastructure providers (Medusa, n.d.). This supports the architectural choice to separate Sako Supply’s account, order, catalog, pricing, and reporting responsibilities. It does not imply that the prototype reproduces Medusa’s production integrations or enterprise functions.

Table 1.2. Documentary comparison of related systems and Sako Supply

System	Documented emphasis	Relevance and scope difference
WooCommerce	WordPress commerce, products, carts, extensibility	Reference for catalog-to-checkout flow; rice-specific rules would require store configuration/customization.

System	Documented emphasis	Relevance and scope difference
Odoo	Integrated web business applications including commerce and warehouse management	Reference for linking customer and administrative functions; broader ERP scope is outside this prototype.
ERPNext	Stock-related order management and accounting tools	Reference for stock/order relationships; Sako Supply provides a limited order and cash-recording workflow.
Medusa	Modular commerce building blocks and customizable API/application structure	Reference for separating application responsibilities; external production providers are not connected in Sako Supply.
Sako Supply	Single bigasan, 25 kg sacks, sample tiers/zones, inventory reservation	Focused academic configuration with clearly labeled payment/map simulations and documented technical tests.

1.7.5 Technical Guidance and Synthesis

OWASP’s Authentication Cheat Sheet distinguishes identity verification from session management and recommends deliberate password and authentication controls (OWASP Foundation, n.d.). The prototype uses salted password hashing, server-managed sessions, failed-login limits, and authorization checks. Nevertheless, its demo administrator password, absence of email verification and multifactor authentication, and limited security evaluation mean that full conformance with production authentication guidance must not be claimed.

Node.js documents the node:sqlite interface used for local SQL access (Node.js contributors, n.d.). The study relies on integer-centavo calculations, parameterized SQL, and transactions for the ordering workflow. The executed last-sack concurrency test provides evidence for a specific inventory behavior in the implementation, not a general guarantee about every deployment or load condition.

The reviewed systems establish that commerce, inventory, and administrative information can be connected. The contribution of this project is a focused, explainable prototype and evaluation plan for the specified rice-ordering context. It is not a claim that existing systems lack the capability to serve rice stores. The five objectives therefore remain the basis for the requirements and procedures developed in Chapter II.

CHAPTER II
METHODOLOGY

2.1 Research and Development Approach

The study uses a development-oriented applied approach with iterative prototyping. Its purpose is to produce a working system and examine whether the implemented workflow meets the stated requirements. The researcher works individually and is responsible for requirements documentation, design decisions, implementation review, testing, and research reporting. Feedback may be requested from an instructor or prospective users, but this does not change the project into a group development activity.

The methodology separates three forms of evidence: documented software requirements, technical validation records, and user-evaluation evidence. The available prototype and its repository provide implementation evidence. Interviews, observation records, and user acceptance forms must be collected through the planned procedures before the study can report findings about an actual store or customer group.

2.2 Development Model

An iterative prototyping model is appropriate because the interface and business rules can be reviewed in small, demonstrable increments. The proposed cycle consists of identifying a requirement, designing the corresponding screen/data flow, implementing it, executing a focused check, and revising it based on the result. The cycle is applied to the customer workflow first and then to the administrator workflow. This approach does not require Scrum roles, a development team, or team ceremonies.

The first increment establishes the catalog, account access, and ordering flow under SO1. The second connects checkout to stock reservation and cancellation under SO2. The third adds order visibility and records under SO3. The fourth adds monitoring and reporting under SO4. The fifth consolidates technical checks and prepares the user evaluation under SO5. This sequence preserves the connection between the research objectives and the development work.

2.3 Requirements Gathering

2.3.1 Documentary Review

The initial requirement set is derived from the supplied research outline, the IT 201 defense rubric, the project’s documented scope, and the current repository. The researcher will maintain a requirement record identifying the originating problem, objective, proposed feature, acceptance condition, and supporting evidence. The related-system review in Chapter I provides design references rather than proof of the needs of a specific local store.

2.3.2 Proposed Interview and Observation

Subject to access and instructor approval, the researcher will seek a prospective bigasan owner or administrator and potential wholesale buyers for requirements validation. A small purposive group, such as one store representative and five to ten potential buyers, may be suitable for formative feedback. These are proposed recruitment targets only; no participants or completed responses are claimed in this draft. The final sample, locale, and dates must be documented after recruitment.

The store interview will ask how rice is described and priced, when stock becomes reserved, how cancellation is handled, how delivery fees are determined, and how cash collection is recorded. Buyer questions will examine the information needed before ordering and the clarity of receipts and order updates. Observation, if permitted, will describe the actual steps and records used in an order without copying unnecessary customer information.

The researcher will compare the validated responses with the prototype's assumptions, including the 25 kg packaging rule, sample tier thresholds, and zone charges. A discrepancy will be recorded as a change request or a limitation, rather than being concealed by treating a demonstration rule as an established business practice.

2.3.3 Consent and Data Handling

Participation will be voluntary and the purpose of the academic exercise will be explained before collecting feedback. Fictional customer details will be used during demonstrations. Any actual interview notes or evaluation records will be restricted to the researcher and authorized academic reviewers. Real payment-card information, wallet credentials, and driver-location data will not be collected for this prototype. Operational backups must also be treated as private because a deployed database can contain customer contact details.

2.4 System Requirements

Table 2.1. Requirements linked to the study objectives

ID / objective	Requirement	Acceptance condition
FR1 / SO1	Catalog, product details, customer access and checkout	Customer can order an available 25 kg rice product; totals are calculated by the server.
FR2 / SO1	Bulk tiers and sample delivery zones	10–24 sacks receive ₱25 off each; 25+ receive ₱50 off each; configured zone fee is added once.
FR3 / SO2	Atomic stock reservation and cancellation	An order cannot exceed available stock; eligible cancellation restores stock once.
FR4 / SO2	Stock movement reasons and reorder points	Admin adjustments include a reason; alerts use each product's configured threshold.
FR5 / SO3	Personal order records, receipts, reorder and updates	Customers can access their own records; receipts reconcile items, discounts, fee and total.

ID / objective	Requirement	Acceptance condition
FR6 / SO4	Order processing, charts, cash ledger and audit	Admin can advance allowed statuses and distinguish completed sales from recorded cash.
NFR1 / SO5	Access control and input handling	Unauthorized access fails; stored markup is rendered as text; image/body limits are enforced.
NFR2 / SO5	Usability and presentation	Main tasks can be attempted on desktop/mobile; navigation, graphics, theme and video can be reviewed.
NFR3 / SO5	Recoverability and traceability	A backup can be restored and checked; test outcomes and admin changes are recorded.

2.5 System Design

2.5.1 Client–Server Architecture

The browser presents the customer and administrative interfaces and submits requests to the Node.js server. The server validates the active session, authorizes the requested operation, calculates prices and fees, and performs parameterized SQL operations against SQLite. Product photos, the tutorial video, and the illustrated map are served locally. The browser is not the authority for payable totals or stock availability.

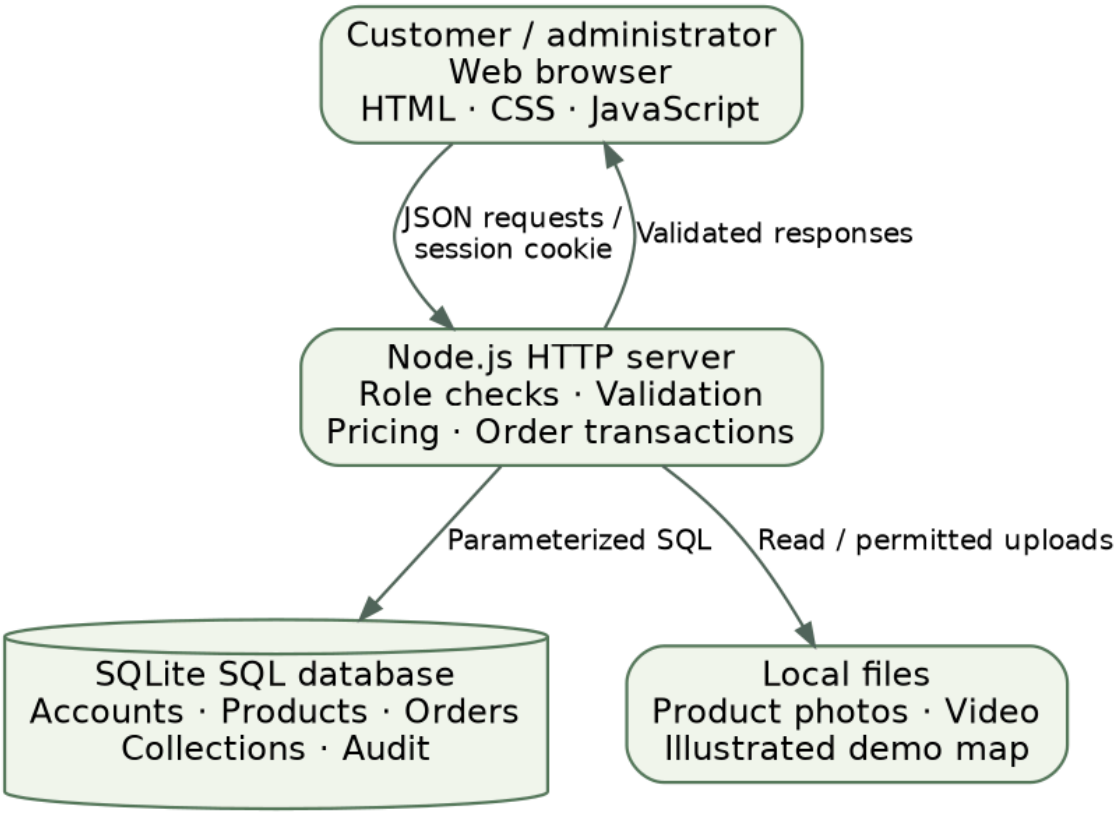


Figure 2.1. Client–server architecture of the implemented prototype.

The architecture supports the five objectives through a shared data layer. The account and order routes support SO1 and SO3; transactional stock updates support SO2; reporting and audit routes support SO4. Validation records and the planned user tasks provide the evidence required by SO5. No external payment processor, SMS service, or rider GPS feed appears in the architecture because those services are not connected.

2.5.2 Use Case Design

The customer initiates catalog, order, receipt, and order-follow-up actions. The administrator manages the catalog and stock, processes orders, records cash collection, and reviews reports. Both roles use the ordinary email/password sign-in interface, but access to admin data is checked on the server. Hiding a dashboard link is a presentation behavior rather than the primary security control.

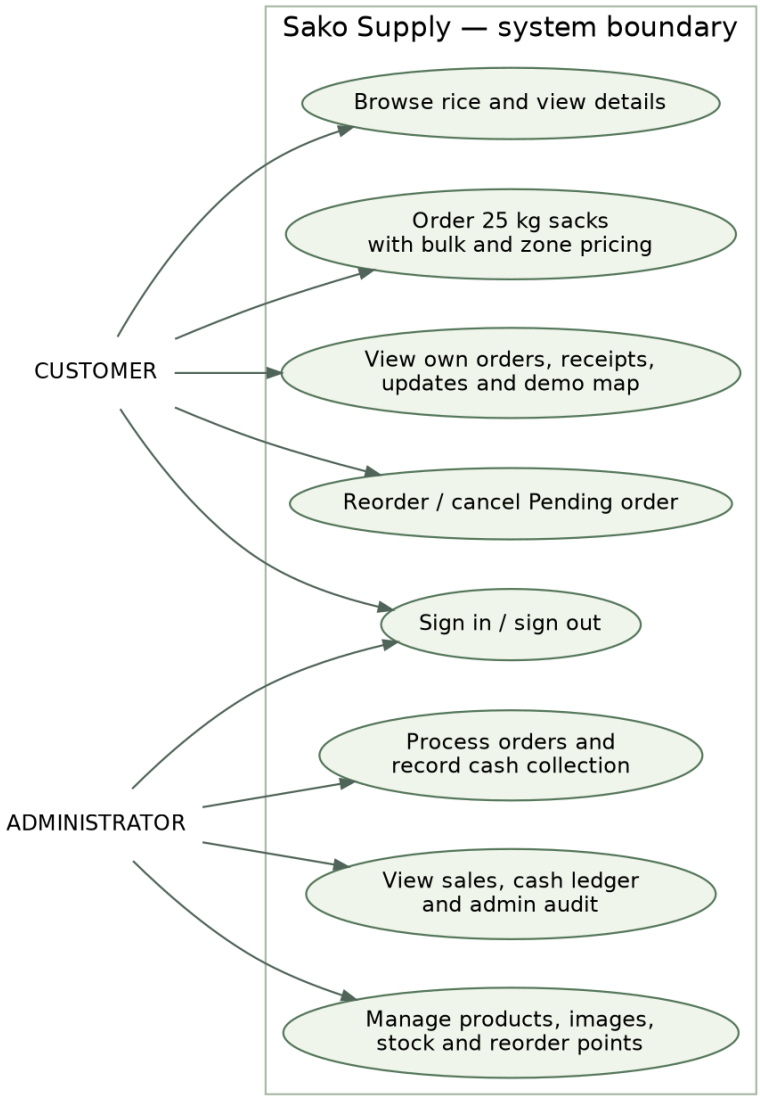


Figure 2.2. Simplified use case view of customer and administrator responsibilities.

2.5.3 Data Flow

At context level, Sako Supply receives account details and order requests from the customer and returns product information, calculated totals, order records, and status information. Administrative input consists of product/stock changes, order-status changes, and cash-collection entries. Administrative outputs consist of order lists, alerts, charts, ledgers, and audit history. Electronic payment outcomes and route positions remain internal demonstration data.

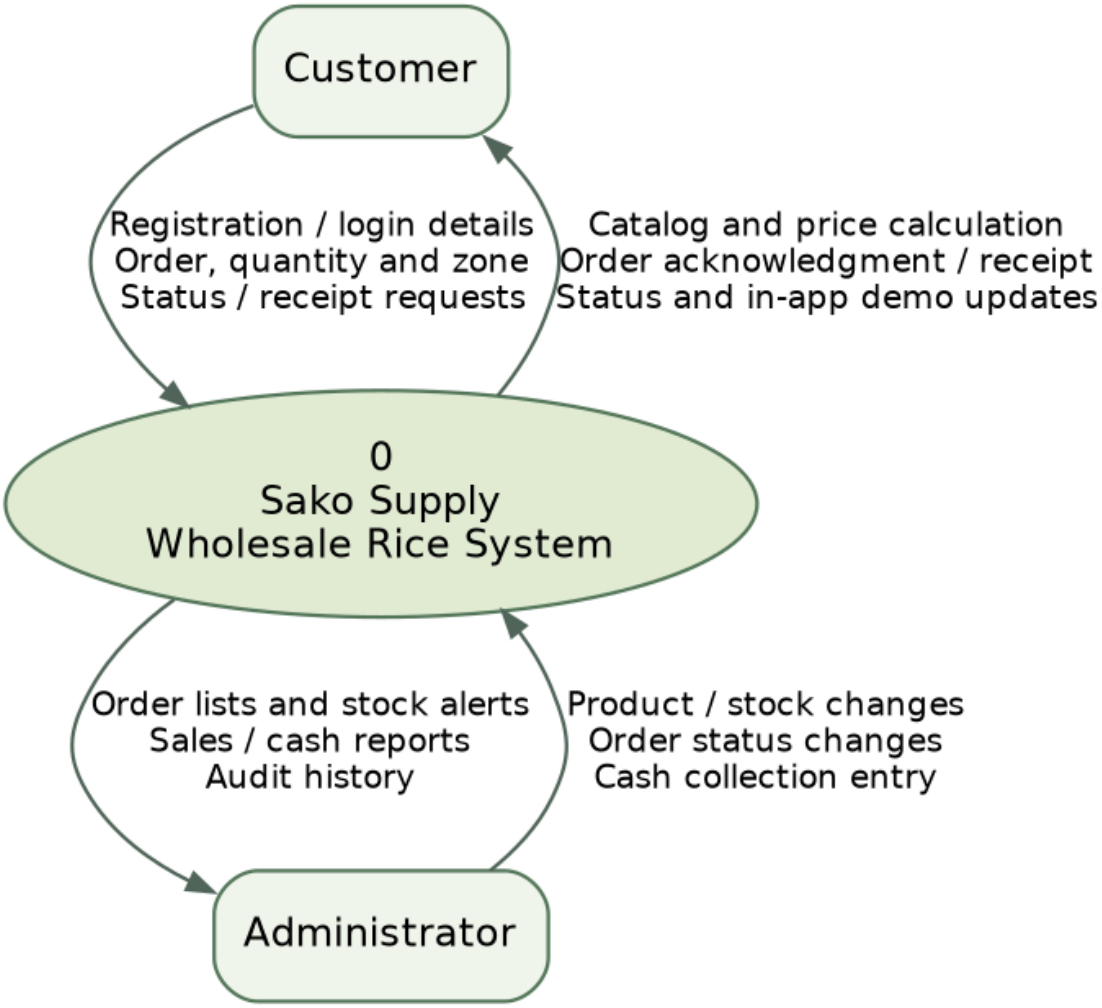


Figure 2.3. Context data-flow diagram for Sako Supply.

2.5.4 Database Design

The database separates customers, products, order headers, order lines, sessions, stock movements, status events, collections, audit records, and notifications. Order items retain name, weight, base-price and effective-price snapshots so later catalog changes do not rewrite a historical receipt. The current implementation keeps an older JSON order snapshot for compatibility; normalized order_items remains the authoritative line-item source.

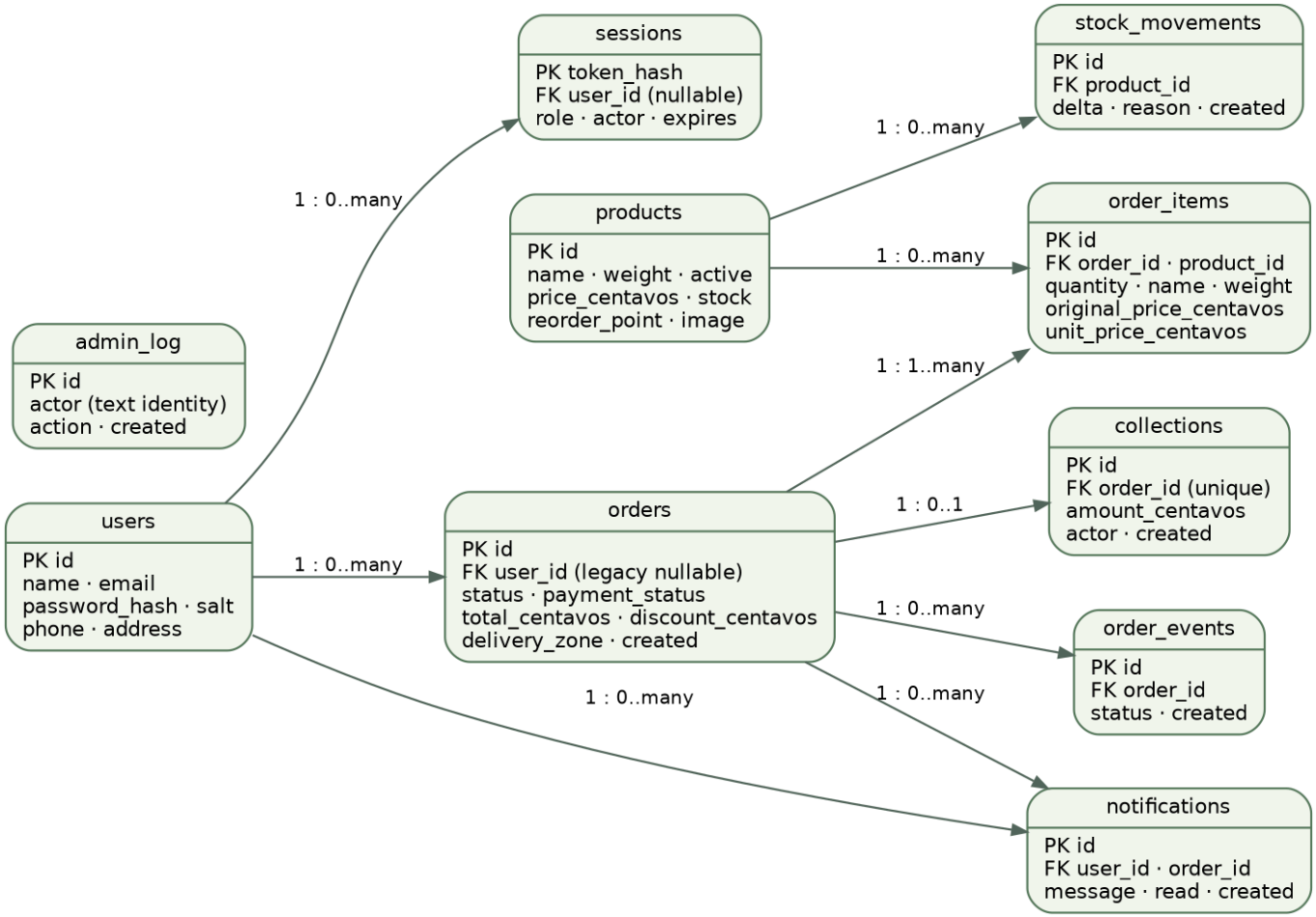


Figure 2.4. Logical entity relationships and selected implementation fields. PK = primary key; FK = foreign key.

The administrator is a configured application identity rather than a customer row. Admin sessions may therefore have a null user_id and an actor email. That actor is copied into the audit and collection records as text, not as a foreign key to users. The figure consequently does not invent a relational link between admin_log.actor and users.id. Legacy orders may also lack a customer user_id, while new customer orders are linked to the authenticated account.

Table 2.2. Principal data entities and their purpose

Entity	Principal key / relationship	Purpose
users	id; unique email	Customer identity, hashed password and basic contact information.
products	id	Rice descriptions, images, stock, reorder point and centavo price.
orders	id; user_id	Order header, status, zone, totals and payment flags.
order_items	id; order_id; product_id	Immutable purchased quantities and base/effective price snapshots.
sessions	token_hash; nullable user_id	Role, actor and expiry for active sessions.
stock_movements	id; product_id	Quantity changes and recorded reasons.
order_events	id; order_id	Order-status history.
collections	id; unique order_id	Recorded cash amount, actor and collection time.
admin_log	id; text actor	Successful admin actions and targets.
notifications	id; user_id; optional order_id	Customer-specific in-app demonstration updates.

2.6 Processing and Business Rules

A cart is priced by total sack quantity across its rice types. For 1–9 sacks, the base price applies. For 10–24 sacks, the sample reduction is ₱25 per sack; at 25 or more, it is ₱50 per sack. A price floor prevents an effective unit price below ₱1. The selected delivery-zone charge is then added, or the delivery charge is zero for pickup. These parameters are demonstration assumptions requiring business validation, not recommendations for a real store’s profit margin.

All pricing arithmetic is performed in integer centavos. For example, ten Dinorado sacks at a base price of ₱1,450 produce a ₱14,500 base subtotal. A ₱250 bulk reduction gives ₱14,250 in rice charges. With the sample Tupi fee of ₱300, the order total becomes ₱14,550. The receipt shows base line amounts and the bulk reduction separately so the discount is applied only once.

The server begins a database transaction before rechecking each requested product and available quantity. A valid order creates its header and item snapshots and reserves the corresponding stock in the same transaction. An invalid request is rejected without a partial reservation. Cancellation is restricted by the current status: a customer can cancel Pending orders, while the administrator controls later eligible transitions. Cancellation restores the reserved quantity only once and records the corresponding event and stock movement.

Delivery orders progress from Pending to Preparing, Out for delivery, and Completed. Pickup orders progress from Pending to Preparing, Ready for pickup, and Completed. Payment status is recorded independently. Completed-order totals therefore differ from the cash-collection ledger. The ledger is based on an explicit admin collection entry, not an assumption that every completed order has been paid.

2.7 Technology Stack and Code Organization

Table 2.3. Technology stack and implementation responsibilities

Layer	Technology / organization	Reason for use
Frontend	HTML, CSS modules and browser JavaScript	Responsive catalog, forms, dialogs, charts, theme controls and print layout without a separate frontend framework.
Backend	Node.js 24; account, order, catalog and report routes	Central server-side validation, authorization and business processing.
Database	SQLite through node:sqlite; db.js migrations	Portable SQL persistence for the individual local-server prototype.
Authentication	auth.js; salted scrypt hashes; server sessions	Customer credential verification and session-role checks.
Business services	Pricing, HTTP-body, security and backup modules	Separate calculations, byte limits, throttling and recovery responsibilities.
Map / media	Leaflet, local illustrated map, local product images and WebM tutorial	Demonstrable visuals without relying on public map tiles.
Validation / versioning	Node test runner, Playwright, Git and GitHub	Repeatable software checks and source/version traceability.
Primary demonstration	Local Node server on a Windows laptop	Demonstration independent of an unverified production deployment.

SQLite is the implemented SQL database; the project is not a MySQL application and does not require XAMPP. Docker and CI configuration files are available as supporting development artifacts. However, the Docker build did not complete in the cloud environment because the container network could not resolve the registry/proxy. The study therefore reports native Node startup and executed checks, not a verified Docker deployment. Likewise, a saved CI workflow is not by itself evidence of a successful remote CI run.

2.8 Testing and Evaluation Plan

2.8.1 Technical Validation

The technical evaluation links each acceptance condition to a repeatable scenario. The recorded development checkpoint reports 19 passing backend/security checks and 11 passing browser checks. The suite exercises business calculations, account ownership, concurrent stock requests, receipt reconciliation, navigation, and other selected behaviors. These results must be rerun on the researcher’s presentation copy after any code or database changes; they are not substitutes for user evaluation.

Table 2.4. Representative technical checks linked to the objectives

Scenario	Expected behavior	Objective
Account ownership	A customer cannot read another customer's order or enter protected admin operations.	SO1, SO3, SO5
Tier and zone calculation	Boundary quantities, mixed rice types, and zone charges produce exact centavo totals.	SO1, SO5
Two-process last-sack race	Two simultaneous attempts for the last sack produce one accepted order and one stock rejection.	SO2, SO5
Cancellation and reorder	Eligible cancellation restores stock once; reorder uses current availability and pricing.	SO2, SO3
Receipt reconciliation	Base amounts minus the bulk reduction plus delivery equal the stored total.	SO3, SO5
Security and robustness	Throttling, expiry, SQL guards, byte limits and stored-markup escaping behave as specified.	SO2, SO5
Charts / collection records	Completed-order charts and date-filtered recorded cash are separate outputs.	SO4, SO5
Navigation / presentation	Product images, theme, mobile views, internal targets and tutorial playback work in tested browser scenarios.	SO3, SO5
Backup recovery	A generated SQL backup opens successfully and preserves the tested data and copied upload fixture.	SO4, SO5

2.8.2 Planned User Acceptance Testing

After requirements validation and technical corrections, the researcher will ask consenting target users to attempt a short, supervised task sequence. Customer tasks will include finding a rice type, reading its details, ordering ten sacks with a delivery zone, reviewing the receipt, and locating order progress. The administrator task sequence will include a stock adjustment, an order-status change, and locating the cash ledger and audit entry.

The proposed record will include task completion, assistance required, critical errors, elapsed time if measured consistently, and open comments. A brief 1–5 rating may be used for clarity of product information, ease of checkout, understanding of the total, usefulness of order records, and navigation. The instrument and participants must be approved or adapted to the instructor's requirements before administration. No rating, average, completion percentage, or participant statement is filled in without an actual record.

For a small purposive group, the analysis will be descriptive. Counts of completed tasks, observed difficulties, and recurring comments will guide revisions. Ratings may be summarized only with their respondent count and limitations. The study will not generalize results to all Philippine rice buyers or claim a causal improvement in business performance from a small formative evaluation.

2.8.3 Alignment with the IT 201 Defense Rubric

The supplied defense format evaluates code quality and organization, meaningful graphics, appearance/design, and navigation/links; its navigation descriptors also mention video playback. The demonstration will therefore show the modular source structure, rice-related images and enlarged views, responsive light/dark presentation, working navigation, and the captioned local tutorial. The panel assigns the grade. Automated checks and this checklist do not guarantee a particular score.

2.9 Chapter Synthesis

The methodology translates the five objectives into concrete requirements and evaluation procedures. SO1 is addressed through catalog and checkout design; SO2 through transactional inventory rules; SO3 through customer records and status visibility; SO4 through administrative monitoring; and SO5 through technical checks and the planned user tasks. Chapter III presents the resulting modules, schedule model, and deliverables using the same identifiers.

CHAPTER III
PROPOSED SYSTEM FEATURES, PROJECT TIMELINE, AND DELIVERABLES

3.1 Overview of the Proposed System

Sako Supply combines a customer storefront and an administrator dashboard around a shared SQL database. The prototype already demonstrates the core workflow, while the broader study still requires completion of the stated field validation and user evaluation. The term proposed system in this chapter identifies the system being presented for academic evaluation; it does not imply that every feature is unimplemented or that the study's empirical phase has already been completed.

The design emphasizes an explainable single-store process: customers order 25 kg sacks, the server calculates the amount and reserves stock, and the administrator processes the order. Customer records and administrative outputs are generated from the same stored transactions. Each module below corresponds to a requirement in Chapter II and an objective in Chapter I.

3.2 Core Modules

3.2.1 Customer Accounts and Product Catalog

Customers use the ordinary sign-in form to access their account-specific features. The server distinguishes customer and administrator roles; guests and customers do not receive protected administrative data. The catalog provides rice names, short descriptions, package weight, prices, and stock indicators. Detail views provide longer descriptions, texture and cooking notes, quantity selection, and an enlarged image view. Generated Sako packaging images are labeled as mockups rather than certified supplier photographs.

3.2.2 Wholesale Checkout and Inventory Control

Customers add sack quantities to a cart and deliberately choose pickup or delivery. For delivery, a supported demonstration zone determines the fee. The server applies the configured quantity tier across rice types and stores the calculation in centavos. Inventory reservation is committed with the order, and cancellation reverses the reserved stock when permitted. Administrators may record stock additions or corrections with a reason and adjust a product's reorder point.

3.2.3 Order Records, Receipts, and Customer Follow-Up

Order history is restricted to the authenticated customer's own account. Customers can open details, inspect the status timeline, print or save the order record as PDF, reorder with current stock/prices, and cancel while Pending. In-app updates demonstrate notification handling without claiming external email or SMS delivery. The map appears for delivery orders and uses simulated movement after dispatch. Pickup orders clearly state that there is no delivery journey to track.

3.2.4 Administrator Dashboard and Reports

The administrator manages products and images, stock movements, reorder points, order processing, recorded cash, and audit history. Daily and weekly charts summarize completed orders from stored data rather than invented sales. Date-filtered reporting distinguishes completed-order value from recorded cash collection. The audit identifies the configured actor, action or target, and time. These functions support monitoring but do not supply cost accounting, staff-role separation, or full financial statements.

3.2.5 Security, Recovery, and Presentation Support

Supporting controls include server authorization, salted customer password hashing, expiring sessions, failed-login limits, order-request limits, input validation, upload signature checks, and a Content Security Policy. The demo administrator password is intentionally not treated as a production-strength credential; the server warns when that default remains in use. Backups use SQLite’s online backup interface and copy uploaded images. The nightly schedule runs only while the server is running.

Presentation support includes responsive layouts, a saved light/dark preference, locally served graphics, and a short captioned ordering video with written steps. A public static app-shell cache provides limited PWA support, but private API responses and order mutations are not cached. These features support the presentation rubric without changing the study’s commercial and evaluation limitations.

3.3 Feature-to-Objective Mapping

Table 3.1. Traceability from objectives to modules and evaluation evidence

Objective	Module / feature	Evidence and current study status
SO1	Accounts, catalog, cart, bulk pricing and zone fees	Implemented; technical checks cover the ordering and calculation workflow. Store rules still require local requirements validation.
SO2	Transactional reservation, cancellation, stock history and reorder points	Implemented; tests cover SQL stock guards, cancellation and competing last-sack requests.
SO3	Private order history, receipts, reorder, updates and demo map	Implemented; browser checks cover records and presentation. Actual user clarity remains subject to planned UAT.
SO4	Product administration, charts, cash ledger and audit	Implemented; monitoring outputs can be demonstrated. Full accounting, credit and staff-role features are excluded.
SO5	Technical suite, planned user tasks and rubric checklist	Technical checks recorded at the stated checkpoint. User acceptance and any institution-required formal evaluation need documented completion.

3.4 Project Timeline and Indicative Gantt Chart

The outline suggests planning, design, development, testing/revision, and documentation/defense phases. Table 3.2 presents an indicative 14-week allocation using the lower ends of the suggested phase ranges. It is a planning model, not a claim that the researcher has already spent fourteen weeks on the project. Because a prototype is available, the researcher must replace this model with documented actual dates and clearly identify the remaining requirements validation and user-evaluation work.

Table 3.2. Indicative full-project schedule; replace with actual dates

Activity	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Planning and requirements	■	■												
System and interface design			■	■										
Iterative development					■	■	■	■	■	■				
Testing and revisions											■	■		
Documentation and defense													■	■

Legend: ■ indicates the illustrative week assigned to an activity. Actual start date: [insert verified date]. Actual completion or target date: [insert verified date]. Requirements interview/observation dates: [insert after collection]. UAT dates: [insert after approval and completion]. The schedule must not be presented as completed work without supporting project records.

3.5 Expected Outputs and Deliverables

Table 3.3. Deliverables and their connection to the research

Deliverable	Connection to Chapters I–II	Status / completion evidence
Working web application prototype	Demonstrates SO1–SO4 through customer and administrator workflows.	Implemented for the documented Node/SQLite local workflow; verify on the presentation laptop.
Source code and database design	Supports code organization and reproducibility under SO5.	Versioned repository, SQL setup/migrations, route/service modules and diagrams are available.
User manual and demonstration materials	Supports understandable ordering, admin operation and navigation.	README, defense guide, captioned local tutorial and written walkthrough are available.
Technical validation record	Provides behavior-based evidence for SO5.	Development record: 19 backend/security and 11 browser checks passed at the stated checkpoint; rerun after changes.
Requirements and UAT records	Validates assumptions and examines intended-user use.	Not supplied for this draft; collect consented evidence using the Chapter II procedures.
Final research document	Connects problems, objectives, methodology and system features.	This editable Chapters I–III draft requires identity/course details, dates, source review, and any actual empirical records before final submission.

3.6 Evaluation and Acceptance of Deliverables

Technical acceptance requires that the documented customer and admin workflows execute without unexplained failures in the selected environment. Critical cases include unauthorized access, a quantity beyond stock, competing attempts for the last sack, incorrect zone or tier calculations, duplicate cancellation, and a receipt total that does not reconcile. An open critical defect in these cases must be reported and corrected before the affected capability is described as verified.

User-facing acceptance will be based on the approved task protocol and documented feedback, not on the presence of a feature alone. A participant who cannot distinguish a demo payment from an actual charge or a simulated route from rider GPS indicates a presentation problem even if the underlying code runs. Findings of this kind will be recorded as revisions and linked back to SO3 and SO5.

3.7 Proposed Future Enhancements

Future work may add cashier/staff permissions, customer credit rules, cost and margin snapshots, multiple saved addresses, complete local-language support, and provider-backed password recovery and email verification. Real card/GCash/Maya payments would require an authorized merchant integration and verified server-side payment events. Real delivery tracking would require an appropriate map/routing service, an actual driver-location source, consent, and operational policies. These extensions are not counted as completed features of the present study.

3.8 Chapter Synthesis

The proposed outputs complete the traceability established in Chapter I and operationalized in Chapter II. Customer ordering addresses information and price consistency; inventory controls address reservation and stock reliability; transaction records address customer visibility; and administrative outputs address monitoring. The documented technical checks support specific implementation claims, while the planned requirements validation and user evaluation remain necessary for conclusions about local usefulness and acceptance. The study should therefore be defended as a working academic prototype with clearly identified evidence and limitations.

REFERENCES

Abdulhakim, H. N. (2026). Sako Supply: A web-based wholesale rice ordering and inventory management system [Unpublished academic prototype; repository checkpoint e711837]. https://github.com/xynvm120/Sako-Supply/tree/e711837af0c6659094afa84392b479a0b944f7ed

Frappe Technologies. (n.d.). ERPNext [Computer software]. GitHub. https://github.com/frappe/erpnext

Medusa. (n.d.). Medusa [Computer software]. GitHub. https://github.com/medusajs/medusa

Node.js contributors. (n.d.). SQLite. In Node.js v24 documentation. https://nodejs.org/docs/latest-v24.x/api/sqlite.html

Odoo S.A. (n.d.). Odoo [Computer software]. GitHub. https://github.com/odoo/odoo

OWASP Foundation. (n.d.). Authentication cheat sheet. OWASP Cheat Sheet Series. https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Authentication_Cheat_Sheet.md

WooCommerce. (n.d.). WooCommerce [Computer software]. GitHub. https://github.com/woocommerce/woocommerce/tree/trunk/plugins/woocommerce

Source note: The related-system descriptions were checked against official project repositories or their official readme material. The supporting Node.js and OWASP documentation was also retrieved for review. These are software/documentary sources, not invented peer-reviewed or local field studies. The researcher should confirm the instructor’s required citation style and add verified local studies if specifically required.

APPENDIX A — ITEMS TO COMPLETE BEFORE FINAL SUBMISSION

• Confirm the researcher’s course/year, adviser, institution details, and actual submission date.

• Identify the real partner store and study locale if there is one; otherwise retain the prototype framing.

• Replace the illustrative Gantt model with documented actual dates and clearly identified remaining activities.

• Conduct or supply the actual requirements interviews/observations, with the required consent records.

• Administer the approved UAT protocol and add only results supported by completed forms or recorded tasks.

• Recheck the references and any instructor-required local literature; do not invent respondents or studies.

• Rerun the software tests and a full demonstration on the laptop/browser used for the defense.

• Keep demo payment, map, receipt, and financial-report limitations visible in both the website and oral presentation.